GRUB (Grand Unified Bootloader)

GRUB is the most common Linux bootloader, located typically in */boot/grub/grub.conf* or */boot/grub2/grub.cfg*. Its responsibilities are:

- Display boot menu (Linux, recovery mode, dual-boot OS).
- Load Linux kernel (*vmlinuz*) into memory.
- Load *initramfs/initrd* (temporary root filesystem with drivers).
- Pass kernel parameters (e.g., root filesystem location).

Kernel loading

GRUB hands control to the Linux kernel. Kernel tasks are:

- Initialise CPU and memory management.
- Load device drivers from *initramfs*.
- Mount the real root filesystem.

Transition to user space

After kernel initialisation, the first user space process (PID 1) is launched:

- Old Linux → */sbin/init*.
- Modern Linux → *systemd*.

The responsibilities are:

- Mount additional filesystems.
- Start essential daemons (networking, logging, etc).
- Bring system to the configured run level/target (multi-user, graphical).

Finally, the login prompt (CLI) or display manager (GUI) appears. At this point, Linux is fully booted and ready for user interaction.

BIOS booting is sequential and limited (MBR, 2TB disk limit, 16-bit real mode). It relies heavily on GRUB/LILO for loading Linux. Modern systems mostly use UEFI, but understanding BIOS boot flow is essential for legacy hardware and OS troubleshooting.

UEFI booting

UEFI booting begins at system power-on, where the firmware initialises hardware and performs integrity checks. Unlike legacy BIOS, which relies on the Master Boot Record (MBR), UEFI loads the operating system's bootloader directly from the EFI System Partition (ESP). This partition is typically formatted as FAT32 and works with modern partitioning schemes such as GPT, supporting larger disks and more partitions.

As systems evolved, BIOS's 16-bit real-mode execution and MBR limitations became a bottleneck. UEFI was designed to:

- Support larger disks (>2TB).
- Provide a modular driver model.
- Include secure booting capabilities.
- Be extensible with pre-OS applications and networking.

Here's how it works.

Power-on and reset

When the power button is pressed**:**

- PSU (power supply unit) receives signal.
- Performs *PWR_OK* check → ensures stable voltages.
- Sends *Power Good* signal to CPU + motherboard.

In the CPU reset state:

- CPU comes out of reset, ready to fetch its first instruction.
- Starts in real mode (16-bit).
  - Backward compatibility with 8086 code.
  - IP (Instruction Pointer) set to reset vector:
    - CS:IP → 0xFFFF:0xFFF0 → Physical address 0xFFFFFFF0.
    - Maps to firmware ROM on motherboard.
    - Firmware places a jump instruction → enters SEC phase.

SEC phase (Security/ Initialisation)

This phase begins with the CPU state setup as follows:

- Clear leftover reset state.
- Disable interrupts.
- Set up a temporary stack (in registers or cache-as-RAM).

Microcode updates come next. CPU microcode fixes are applied from firmware blobs.

Platform security is ensured by verifying digital signatures of the next firmware stage (if Secure Boot, Boot Guard, TPM enabled). Establishing *Root of Trust* ensures execution of trusted code only.

Memory unavailable problem indicates:

- DRAM not yet initialised.
- Use Cache-as-RAM (CAR): CPU cache configured to act as temporary RAM.
- Stack + variables stored in cache.

To locate next stage:

- SEC prepares handoff structure (CPU state, basic platform information).
- Transfers control to PEI phase.

PEI phase (Pre-EFI initialisation)

This phase is run by:

- PEI Core = supervisor.
- PEIMs (Pre-EFI Modules) = workers for CPU, DRAM, chipset.

Key actions are:

- Run on Cache-as-RAM.
- Initialise DRAM:
  - Reads DIMM SPD information.
  - Sets up memory controller (timing, training).
  - Switches stack + data from CAR → real DRAM.
- Initialise essential chipset parts:
  - Clock generators, PCI root basics, debug console
- Build HOBs (Hand-Off Blocks):
  - Memory map
  - CPU + firmware information
  - Firmware volume locations

In the handoff, PEI passes control + HOB list to DXE Core.

DXE (Driver Execution Environment) phase

This phase is run by:

- DXE core = central coordinator
- DXE drivers = handle hardware devices

Key actions are: